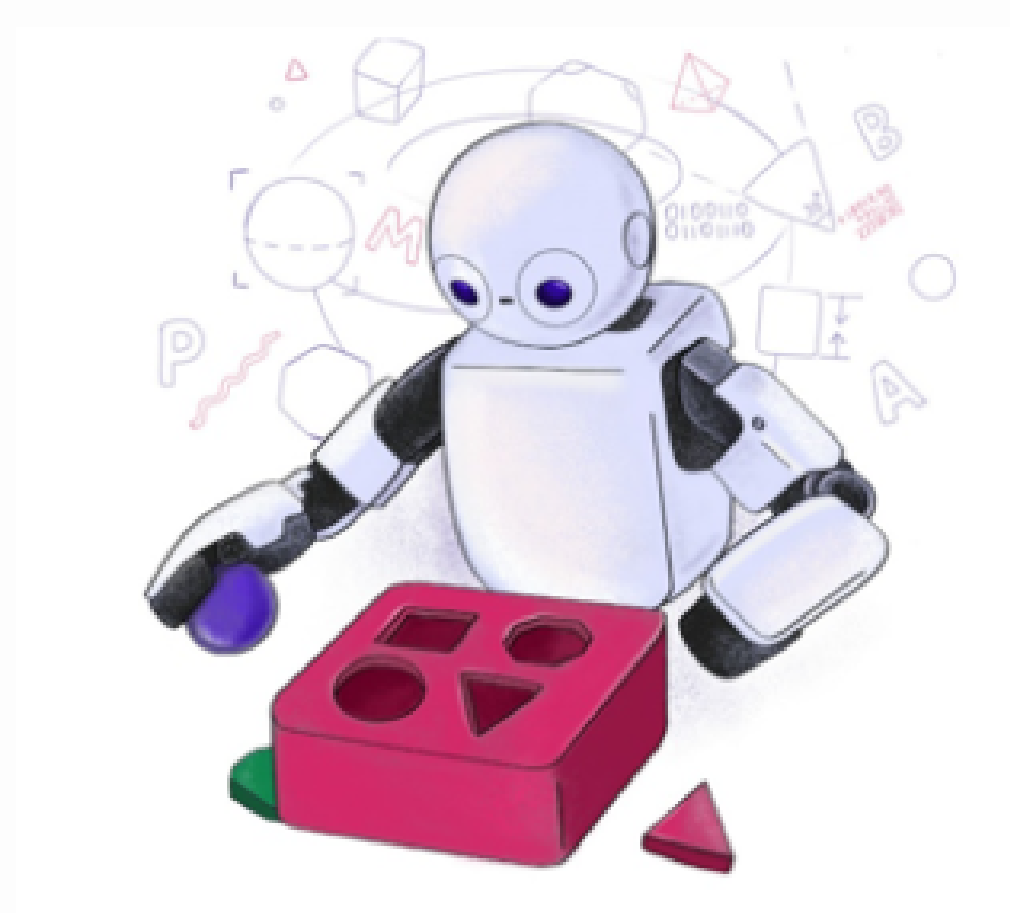


TP558 - Tópicos Avançados em Machine Learning:

XGBoost: A Scalable Tree Boosting System



Contexto do tema

- O Machine Learning está presente em diversas aplicações do mundo real.
- Essas aplicações dependem de modelos precisos e sistemas escaláveis.
- O aumento do volume de dados tornou a eficiência computacional um fator essencial.

Problema geral da área

O Tree Boosting oferece alta precisão em diversas aplicações.

Entretanto, implementações tradicionais possuem limitações de escalabilidade.

Grandes volumes de dados exigem maior eficiência computacional.

Motivação do Artigo

- Desenvolver um sistema de Tree Boosting altamente escalável.
- Tornar o treinamento mais rápido e eficiente.
- Manter alto desempenho preditivo mesmo em grandes volumes de dados.

Objetivo

- Apresentar o XGBoost, um sistema escalável para Tree Boosting.
- Melhorar o desempenho computacional do treinamento.
- Manter alta precisão mesmo em grandes volumes de dados.

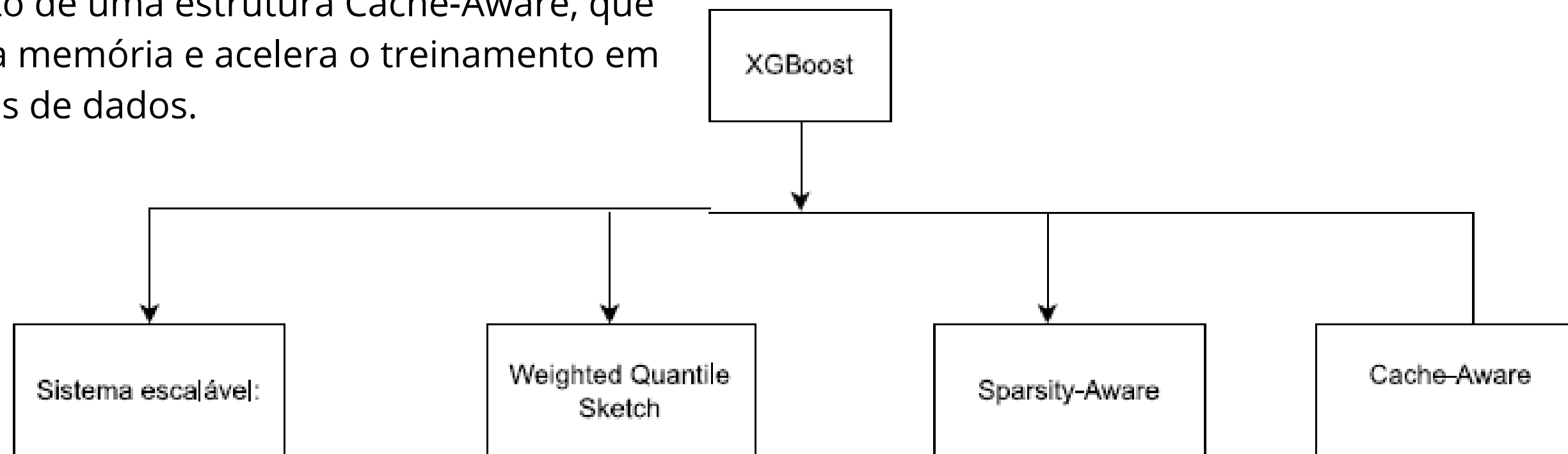
Hipótese

Os autores partem da ideia de que a combinação de otimizações algorítmicas e computacionais pode tornar o Tree Boosting significativamente mais escalável e eficiente, sem comprometer a qualidade das previsões.

Contribuições dos autores

Os autores destacam quatro contribuições principais:

- Desenvolvimento de um sistema altamente escalável para Tree Boosting.
- Proposta do método Weighted Quantile Sketch para cálculo eficiente dos pontos de divisão.
- Introdução de um algoritmo Sparsity-Aware, capaz de tratar eficientemente dados esparsos.
- Desenvolvimento de uma estrutura Cache-Aware, que otimiza o uso da memória e acelera o treinamento em grandes volumes de dados.



Fundamentação Teórica

- Árvore de Decisão
- Tree Boosting
- XG Boost

Trabalhos relacionados mais importantes

Trabalho	Contribuição
Gradient Tree Boosting (Friedman)	Base do XGBoost.
CART	Estrutura das árvores de decisão utilizadas pelo modelo.
LambdaMART	Variante do Tree Boosting para problemas de ranqueamento.
Tree Boosting Paralelo	Os autores mostram que existiam trabalhos anteriores, mas nenhum integrava cache-aware, sparsity-aware e out-of-core.

Metodologia

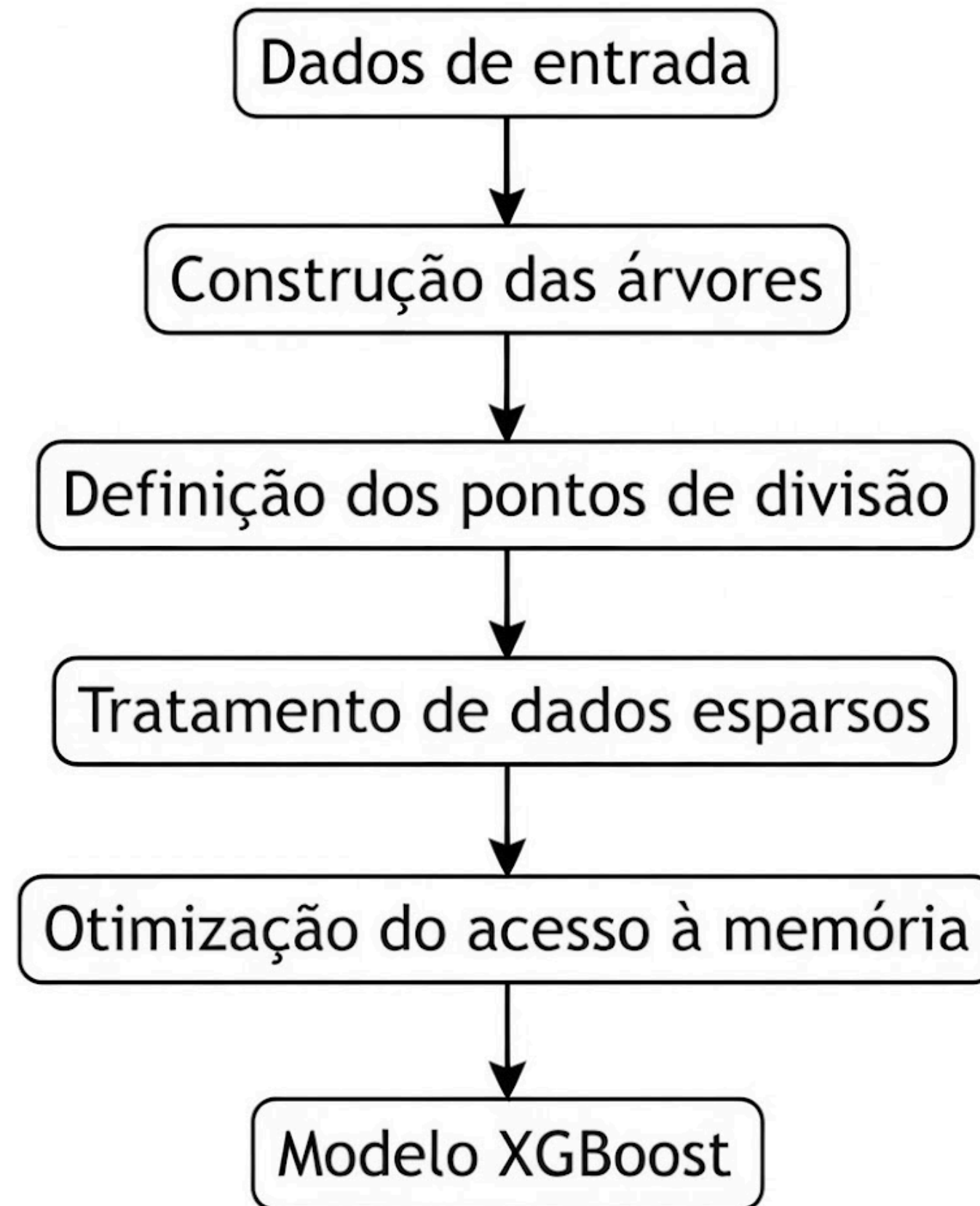
Base de dados / cenário experimental

Aspeto	Descrição
Conjuntos de dados	Diferentes bases de dados utilizadas em tarefas de classificação, regressão e ranqueamento.
Comparação	XGBoost comparado com outras implementações de Tree Boosting.
Cenários de execução	Testes em memória (in-memory), distribuído e out-of-core.
CrITÉrios de avaliação	Precisão, tempo de treinamento e escalabilidade.

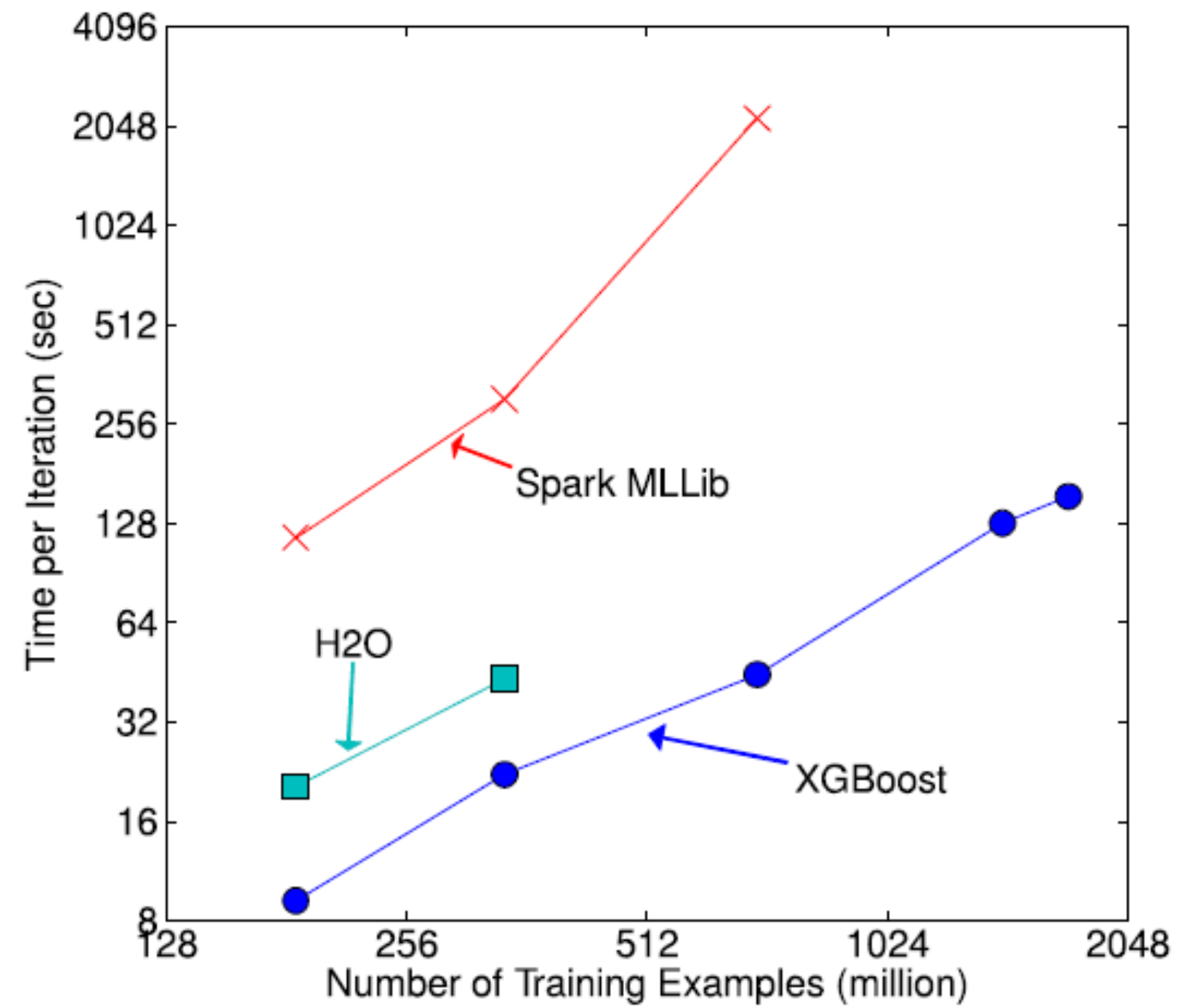
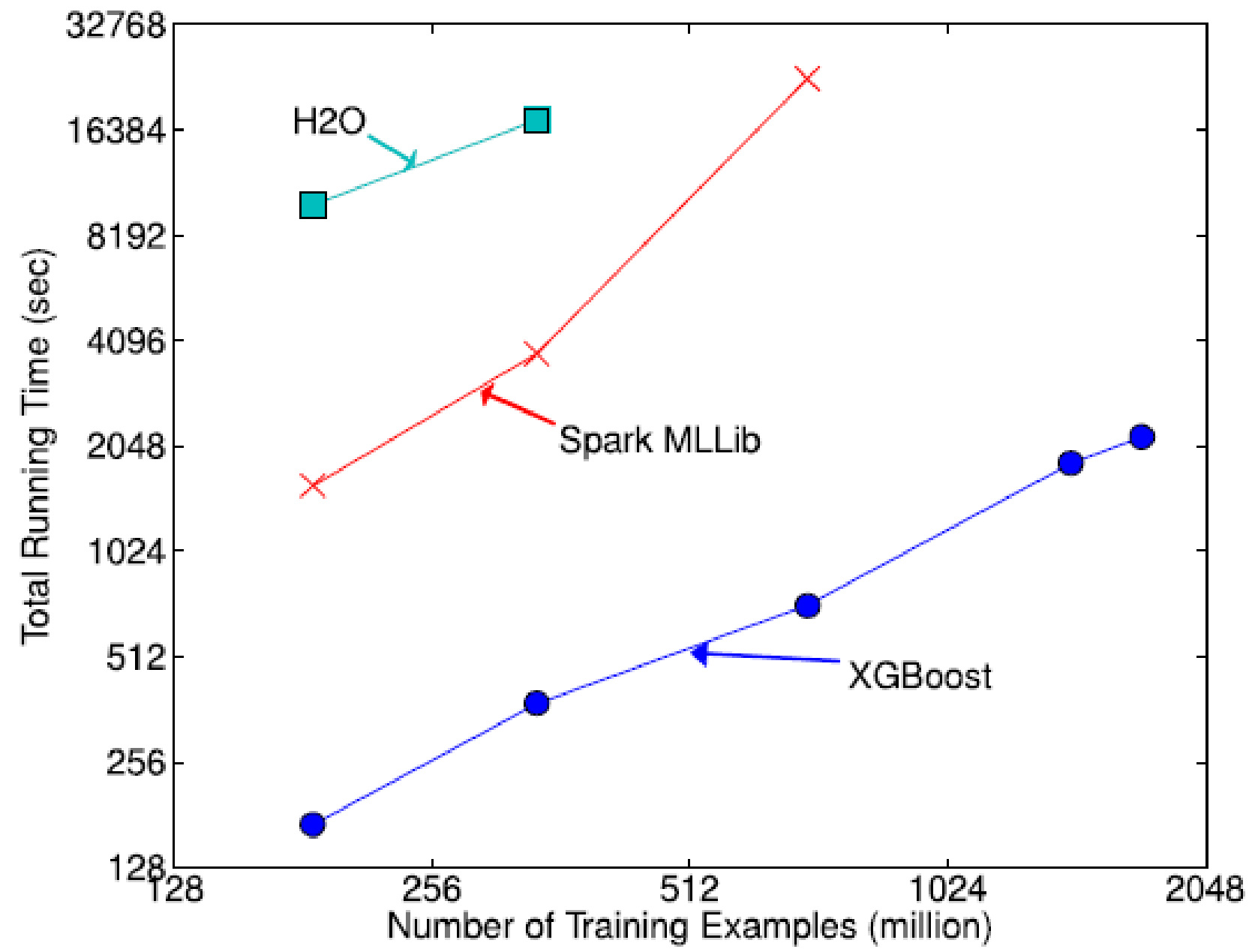
Abordagem proposta pelo XGBoost

Técnicas	Função
Exact Greedy	Encontra a melhor divisão dos dados.
Approximate Algorithm	Acelera o treinamento.
Weighted Quantile Sketch	Melhora o processamento dos dados.
Sparsity-Aware	Trata dados esparsos automaticamente.
Cache-Aware	Otimiza o uso da memória.

Fluxo do método



Resultados



O que funcionou

- Menor tempo de execução.
- Alta escalabilidade.
- Bom desempenho em uma única máquina e em sistemas distribuídos.
- Processamento eficiente de grandes volumes de dados.

O que não funcionou

Os autores não relataram falhas relevantes nos experimentos apresentados por eles.

Reprodução parcial dos resultados

```
Testar o modelo

[8] ✓ 0s # Fazer previsões
previsoes = modelo.predict(X_teste)

# Calcular a precisão
precisao = accuracy_score(y_teste, previsoes)

print(f"Precisão: {precisao*100:.2f}%")

... Precisão: 96.49%

[7] ✓ 0s
import time
import xgboost as xgb

inicio = time.time()

modelo = xgb.XGBClassifier(random_state=42)
modelo.fit(X_treino, y_treino)

fim = time.time()

print("Tempo de treinamento:", fim - inicio, "segundos")

Tempo de treinamento: 0.12076997756958008 segundos
```



Análise crítica

Pontos fortes do trabalho

- Elevada precisão preditiva.
- Baixo tempo de treinamento.
- Boa escalabilidade para grandes volumes de dados.
- Tratamento eficiente de dados esparsos e valores ausentes.

Limitações do trabalho

- Pode exigir muitos recursos de memória em alguns cenários.
- Necessita de uma configuração adequada para alcançar o melhor desempenho.

Possíveis melhorias

- Tornar o algoritmo mais simples de configurar.
- Reduzir ainda mais o consumo de memória.

Aplicações práticas

- Detecção de fraudes.
- Classificação de textos.
- Diagnóstico médico.
- Previsão de vendas.
- Sistemas de recomendação.
- Análise financeira.

Conclusão

Síntese do trabalho

- O artigo apresentou o XGBoost como um sistema escalável para Tree Boosting.
- As otimizações propostas melhoram a eficiência computacional sem comprometer a qualidade preditiva.
- Os experimentos demonstraram a eficácia da abordagem em diferentes cenários.

Impacto do trabalho

- Tornou o Tree Boosting mais rápido e escalável.
- Tornou-se uma das ferramentas mais utilizadas em Machine Learning.
- É utilizado em empresas, pesquisas e competições.

Direções futuras

- Aprimorar continuamente a escalabilidade e o desempenho computacional.
- Explorar novas arquiteturas de hardware e ambientes distribuídos.
- Aplicar o XGBoost em novos problemas e áreas de Machine Learning.

Perguntas ?

Referências

- CHEN, T.; GUESTRIN, C. XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), New York: ACM, 2016, p. 785–794

Obrigada!

Quiz:

https://docs.google.com/forms/d/e/1FAIpQLSeKINam7QDRMKr3bb_ULBoXlv5XJDPzpf5sMdm24BxC_M2CPw/viewform?usp=publish-editor